
Processador: Caminho de Dados e Controle

Capítulo 5 (Patterson e Hennessy, Organização e Projeto de Computadores – 3ª edição)

Instruções:

- Linguagem de Máquina
- MIPS
 - Microprocessor without Interlocked Pipeline Stages
 - RISC (Reduced Instruction Set Computer)
 - Instruções possuem 3 operandos
 - Ordem dos operandos fixada (primeiro o destino)

Exemplo:

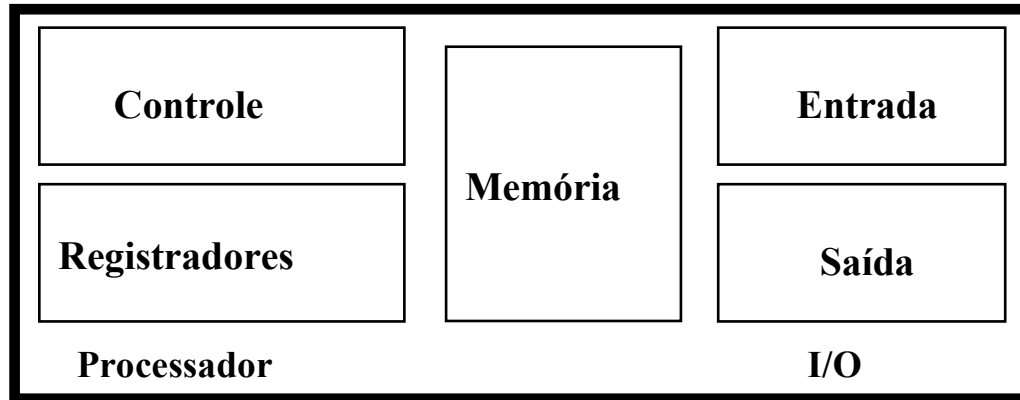
código C: $A = B + C$

código MIPS: `add $s0, $s1, $s2`

(registradores são associados na compilação)

Registradores vs. Memória

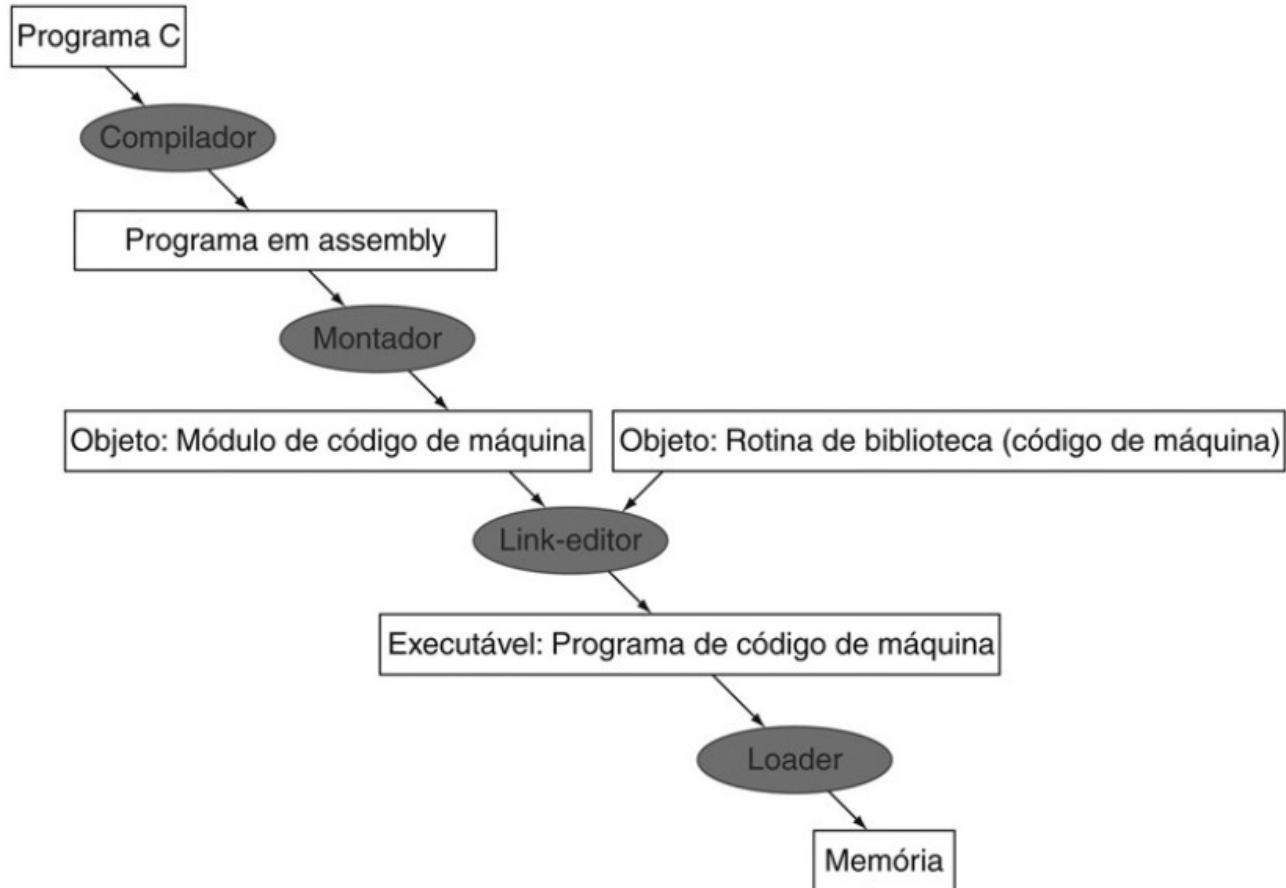
- **Instruções Aritméticas opera sobre registradores**
 - apenas 32 registradores
- **Compilador associa variáveis com registradores**
 - programas com muitas variáveis?
 - **Memória de 32 bits**
 - **Palavras de 4 bytes (2^{30} posições)**



Convenção de Registradores

Nome	Número do registrador	Uso	Preservado na chamada?
\$zero	0	O valor constante 0	n.a.
\$v0-\$v1	2-3	Valores para resultados e avaliação de expressões	não
\$a0-\$a3	4-7	Argumentos	não
\$t0-\$t7	8-15	Temporários	não
\$s0-\$s7	16-23	Valores salvos	sim
\$t8-\$t9	24-25	Mais temporários	não
\$gp	28	Ponteiro global	sim
\$sp	29	Stack pointer	sim
\$fp	30	Frame pointer	sim
\$ra	31	Endereço de retorno	sim

Compilador vs Montador



Instruções Básicas

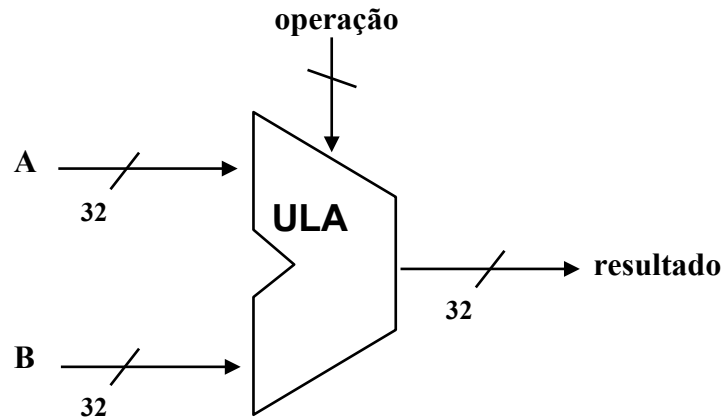
- | Instrução | Significado |
|-------------------------------------|--|
| <code>add \$s1,\$s2,\$s3</code> | <code>\$s1 = \$s2 + \$s3</code> |
| <code>sub \$s1,\$s2,\$s3</code> | <code>\$s1 = \$s2 - \$s3</code> |
| <code>lw \$s1,100(\$s2)</code> | <code>\$s1 = Memory[\$s2+100]</code> |
| <code>sw \$s1,100(\$s2)</code> | <code>Memory[\$s2+100] = \$s1</code> |
| <code>bne \$s4,\$s5,LPróxima</code> | <code>instr. está em L se \$s4 ≠ \$s5</code> |
| <code>beq \$s4,\$s5,LPróxima</code> | <code>instr. está em L se \$s4 = \$s5</code> |
| <code>j Label</code> | <code>Próxima instr. está em Label</code> |

- Formatos:

R	op	rs	rt	rd	shamt	funct
I	op	rs	rt	16 bit address		
J	op	26 bit address				

Unidade Lógica e Aritmética (ULA)

- Operações básicas
 - Soma, subtração, etc.
 - AND, OR, XOR, etc.
- Resultado em um único ciclo

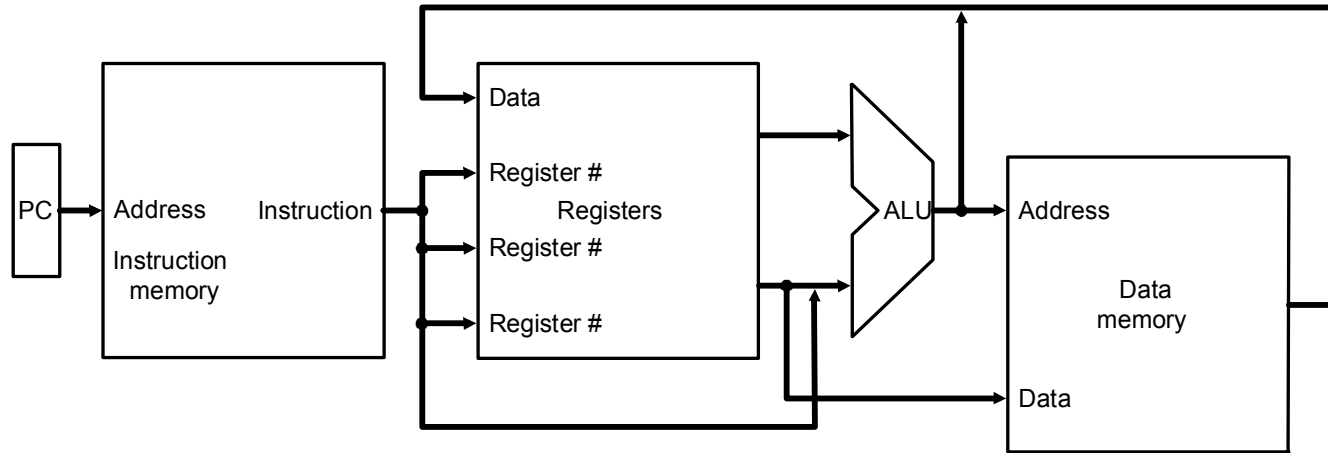


Caminho de Dados e Controle

- **Implementação do MIPS**
- **Simplificada para conter apenas:**
 - Instruções de referência a memória: `lw, sw`
 - Instruções lógicas e aritméticas: `add, sub, and, or, slt`
 - Instruções de controle de fluxo: `beq, j`
- **Implementação genérica:**
 - Program Counter (PC) indica endereço das instruções
 - Lê instrução na memória
 - Lê os registradores envolvidos
 - Com base na instrução decide o que fazer
- **Todas instruções utilizam a ULA após leitura dos registradores**
referência a memória, aritmética, controle de fluxo

Circuitos sequencias e combinacionais

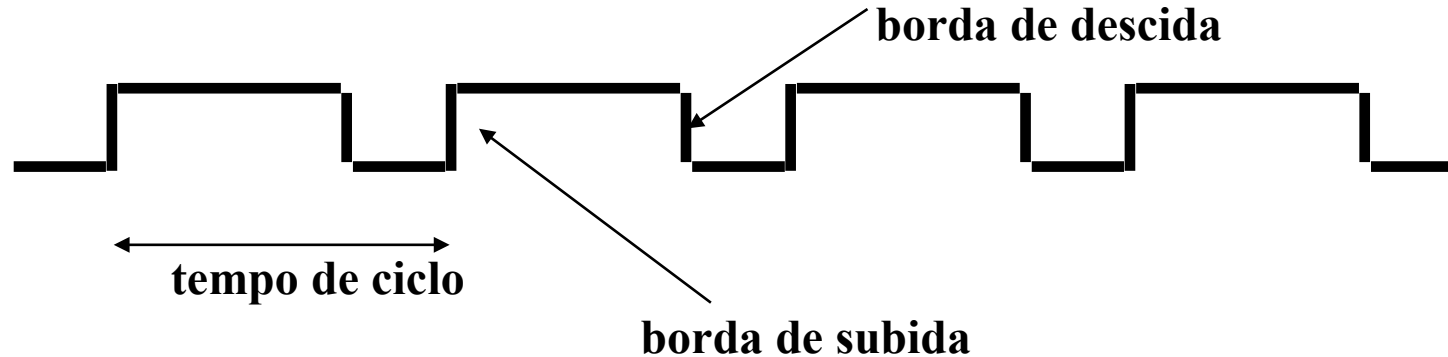
- **Visão Simplificada:**



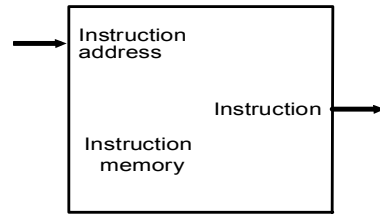
- **Dois tipos de unidades funcionais:**
 - Elementos que operam sobre dados (combinacionais)
 - Elementos que contêm estados (sequenciais)

Elementos de Estado

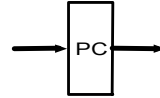
- **Memória**
 - Mantém a saída mesmo quando a entra é alterada
- **Sincronismo com o Clock**
 - Sensível a borda de subida
 - Sensível a borda de descida



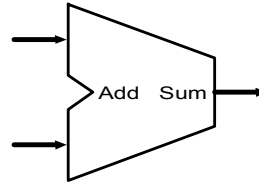
Unidades Funcionais



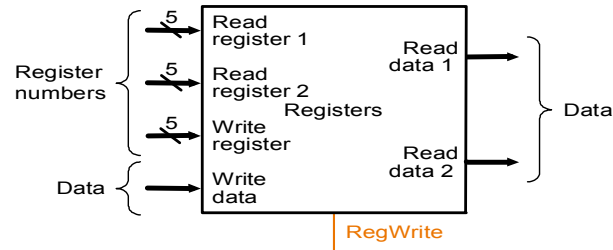
a. Instruction memory



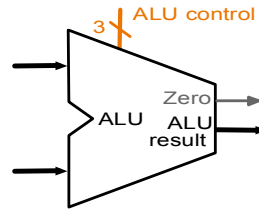
b. Program counter



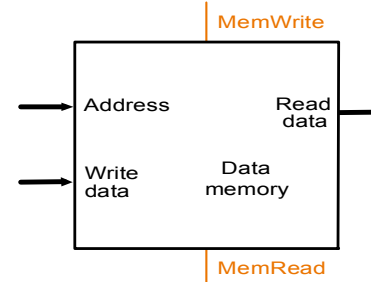
c. Adder



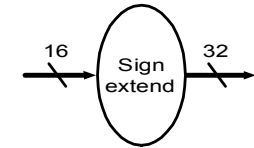
a. Registers



b. ALU



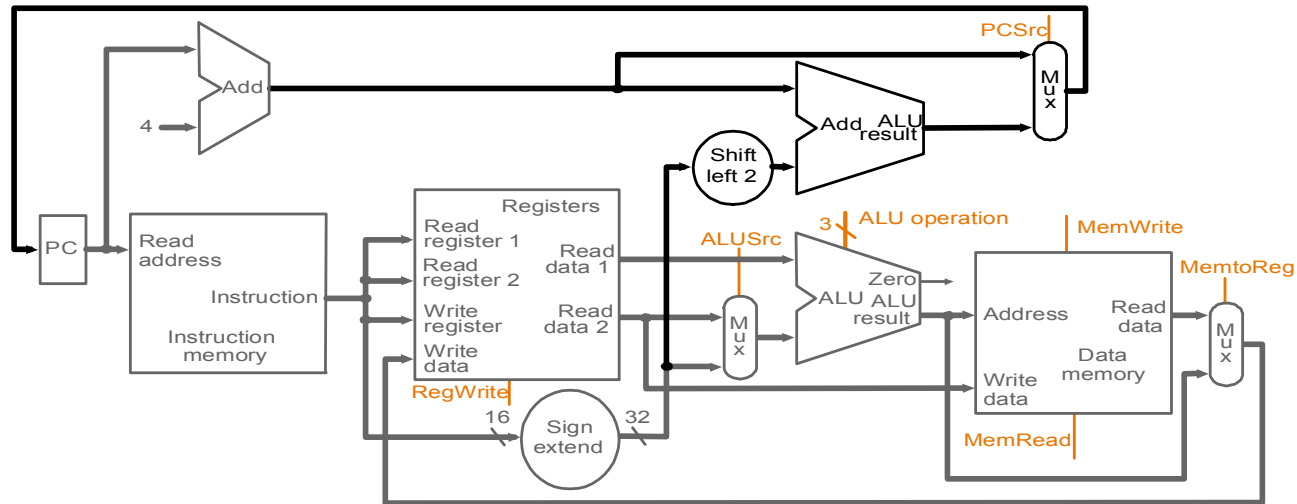
a. Data memory unit



b. Sign-extension unit

Caminho de Dados

- Multiplexadores direcionam os dados



Controle Monociclo

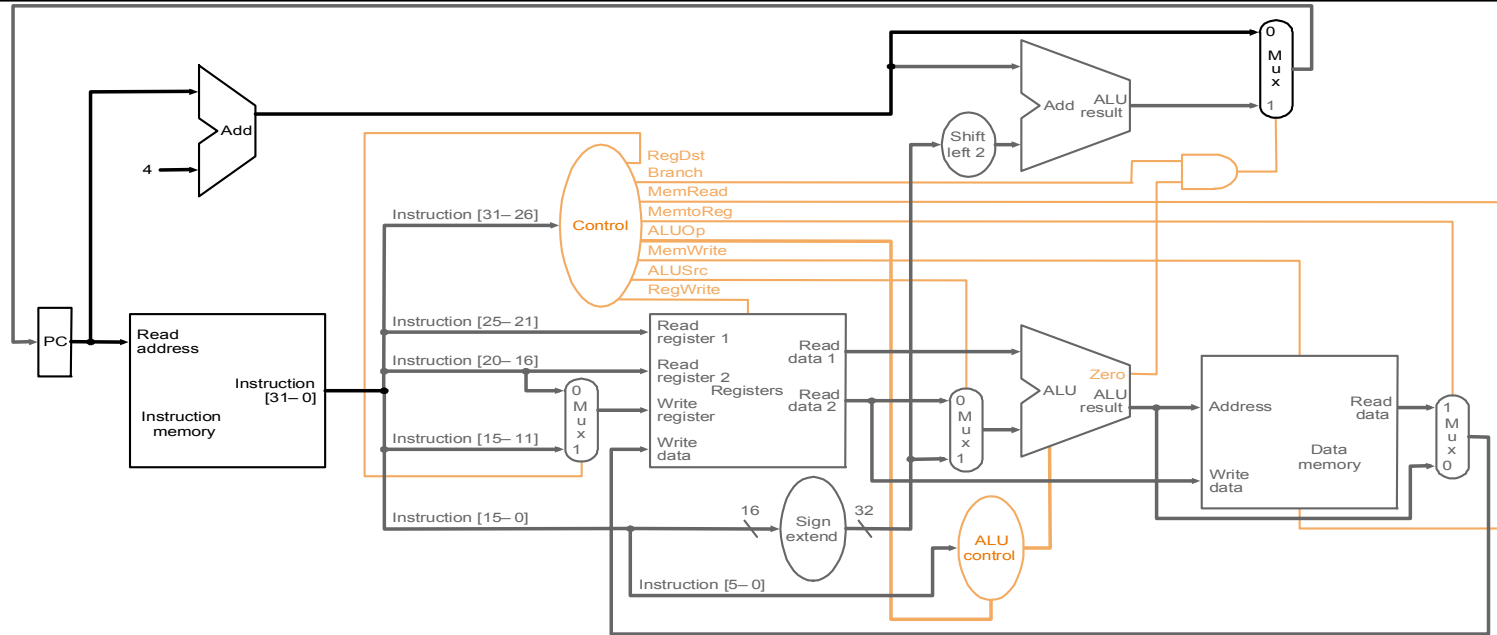
- **Seleciona as operações a serem executadas (ULA, read/write, etc.)**
- **Controla o fluxo de dados (multiplexadores)**
- **Informação de controle está na instrução de 32 bits**
- **Exemplo: add \$8, \$17, \$18**
- **Formato da Instrução:**

000000	10001	10010	01000	00000	100000
--------	-------	-------	-------	-------	--------

op	rs	rt	rd	shamt	funct
----	----	----	----	-------	-------

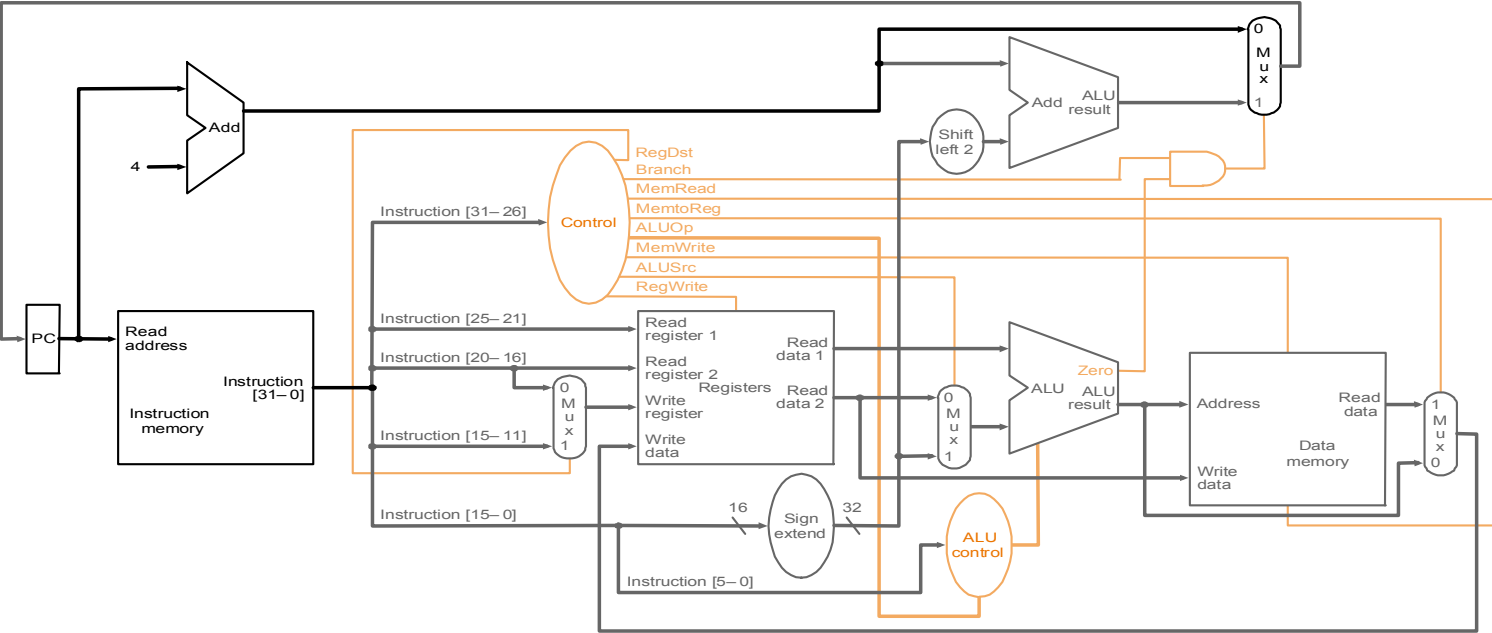
- **Operação da ULA baseado no tipo da instrução e código da função**

Controle Monociclo



Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUp0
R-format	1	0	0	1	0	0	0	1	0
lw									
sw									
beq									

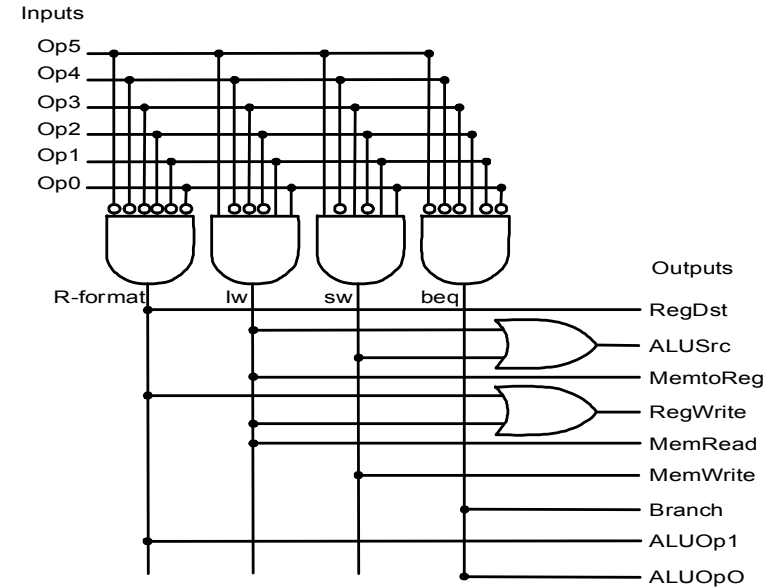
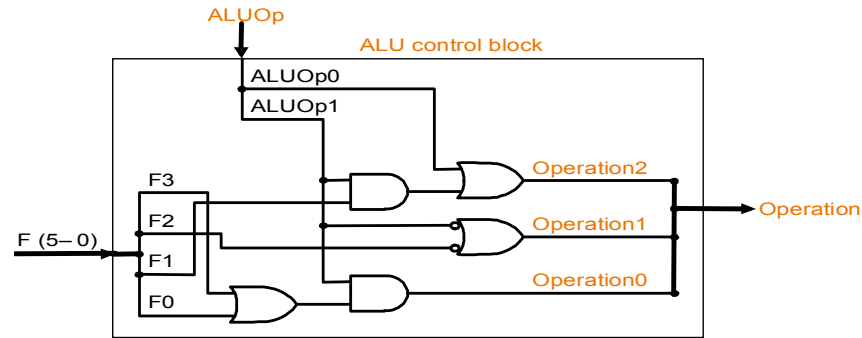
Controle Monociclo



Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

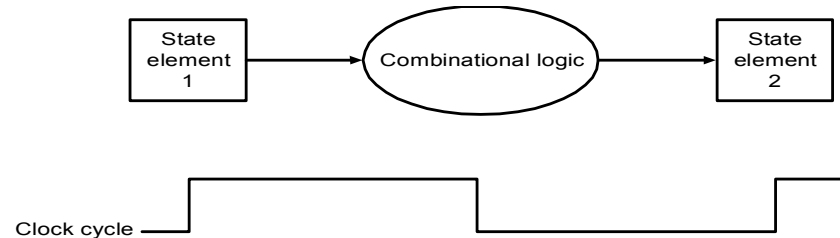
Controle Monociclo

- Lógica Combinacional (tabelas verdades)



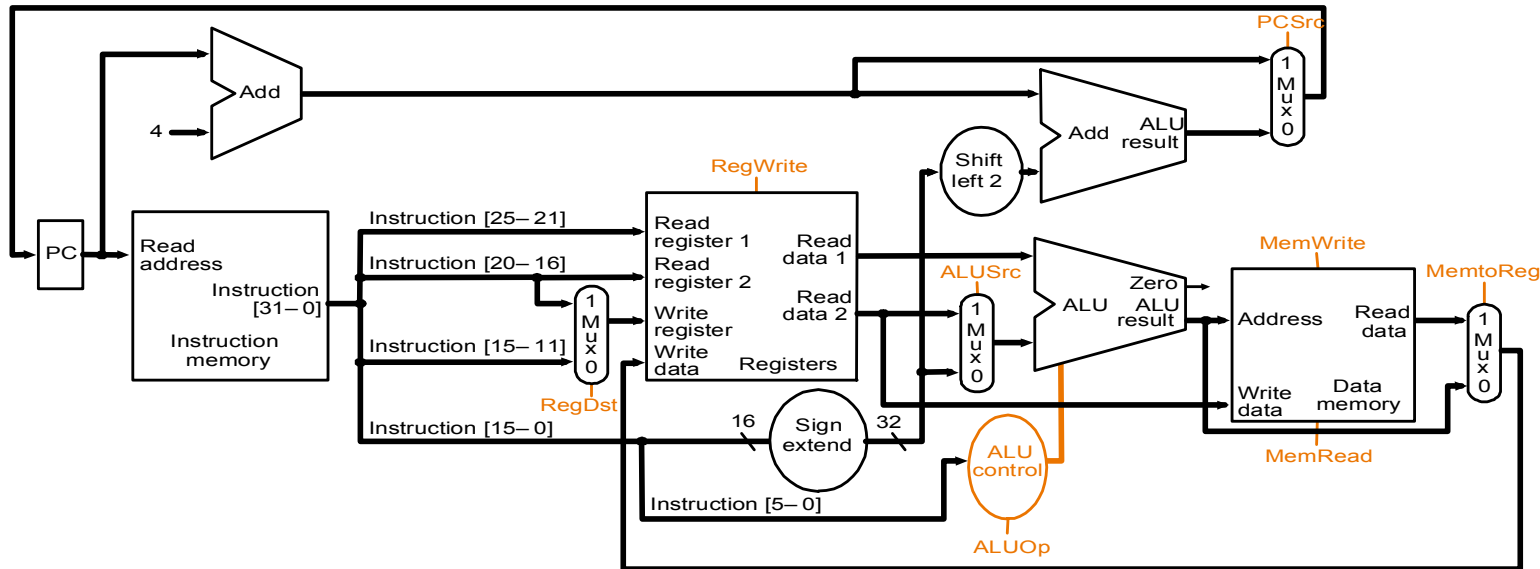
Controle Monociclo

- Toda lógica é combinacional
- Espera a informação se propagar por todo circuito
- Existe propagação dentro da ULA, memória, etc.
- Tempo de Ciclo determinado pelo caminho mais longo de propagação



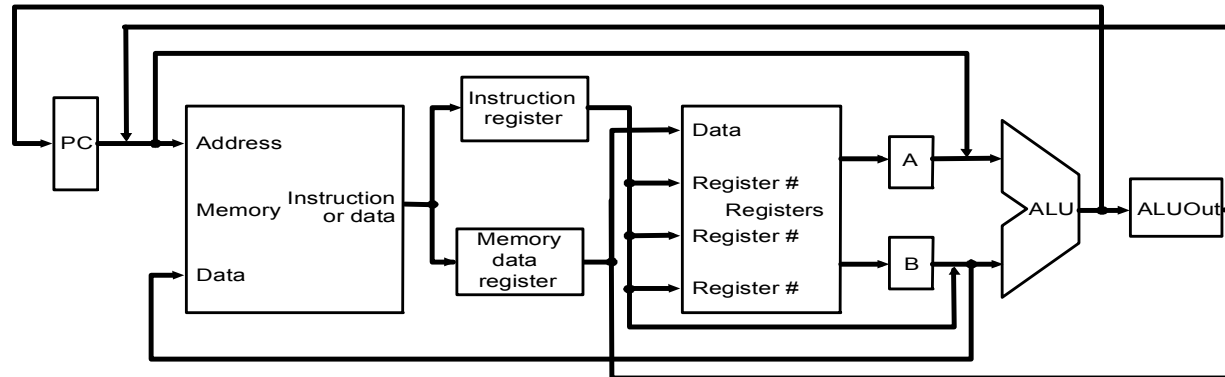
Controle Monociclo

- Calcule o tempo de ciclo assumindo atraso desprezível nas unidades, com exceção de:
 - memória (2ns), ULA (2ns), registradores (1ns)
- Qual o caminho de dados para as instruções: 1) add \$t1, \$t2, \$t3; 2) lw \$t1, 100(\$t2); 3) beq \$t1, \$t2, 50



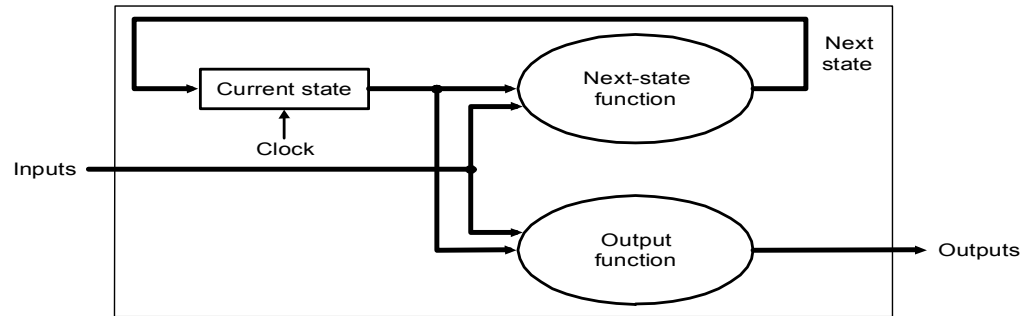
Controle Multiciclo

- **Problema com controle monociclo:**
 - E se existe uma instrução complicada (ponto flutuante)?
 - Desperdício do tempo de ciclo
- **Solução:**
 - Usar um tempo de ciclo menor
 - Diferentes instruções usam diferentes quantidades de ciclo
 - Um caminho de dados multiciclo
 - Registradores especiais entre ciclos



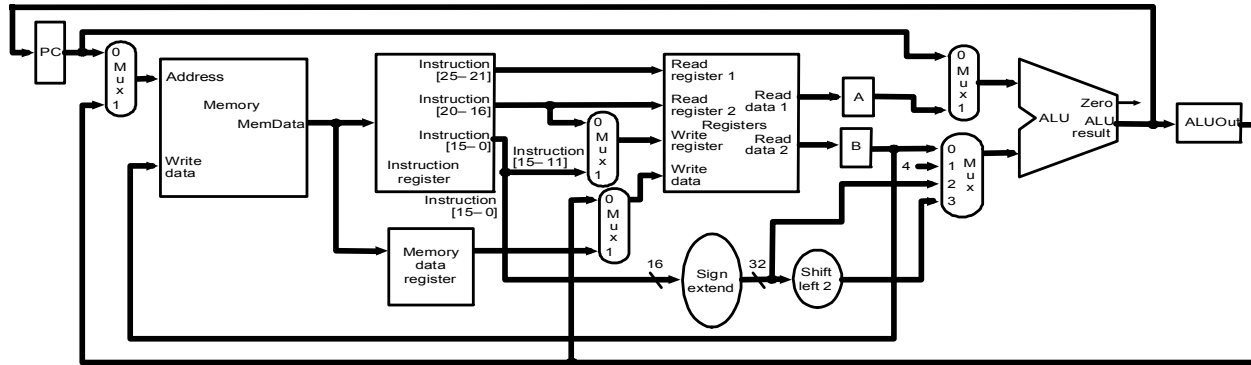
Controle Multiciclo

- **Reuso de Unidades Funcionais**
 - ULA utilizada para computar endereço, incrementar PC, operações lógicas e aritméticas
 - Memória utilizada para instruções e dados
- **Controle feito por uma máquina de estado finita**



Controle Multiciclo

- Instruções divididas em passos
 - Cada passo leva um ciclo de tempo
 - Diminuir o desperdício por ciclo
 - Cada ciclo usa apenas uma unidade funcional complexa (memória, registradores, ULA)
- No fim do ciclo
 - Armazena os valores para serem utilizados no próximo ciclo
 - Registradores Especiais



Execução em 5 passos

- **Busca da Instrução**
- **Decodificação da Instrução e busca do registrador**
- **Execução, cálculo do endereço da memória ou efetivação do desvio condicional**
- **Acesso a memória ou escrita em registradores**
- **Escrita em Registrador**

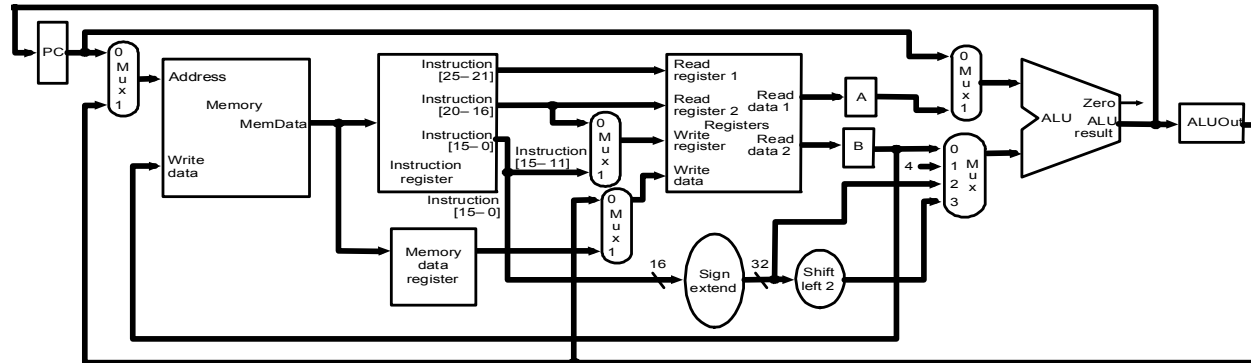
INSTRUÇÕES LEVAM DE 3 - 5 CICLOS!

Passo 1: Busca da Instrução

- Usa o PC para carregar instrução em IR
- Incrementa 4 no PC e coloca o resultado no PC

$IR = \text{Memory}[PC];$

$PC = PC + 4;$

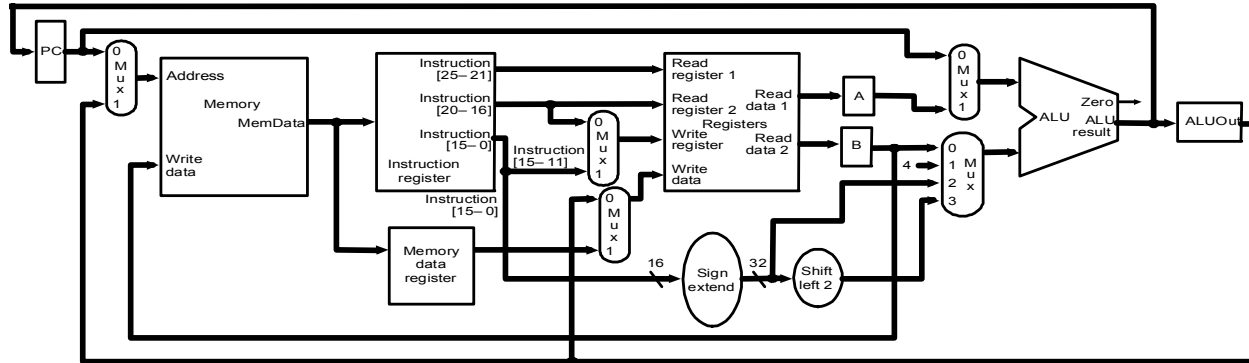


Passo 2: Decodificação e Busca no Registrador

- Lê registradores rs e rt caso ele seja necessário
- Calcula o endereço de desvio caso seja uma instrução de desvio

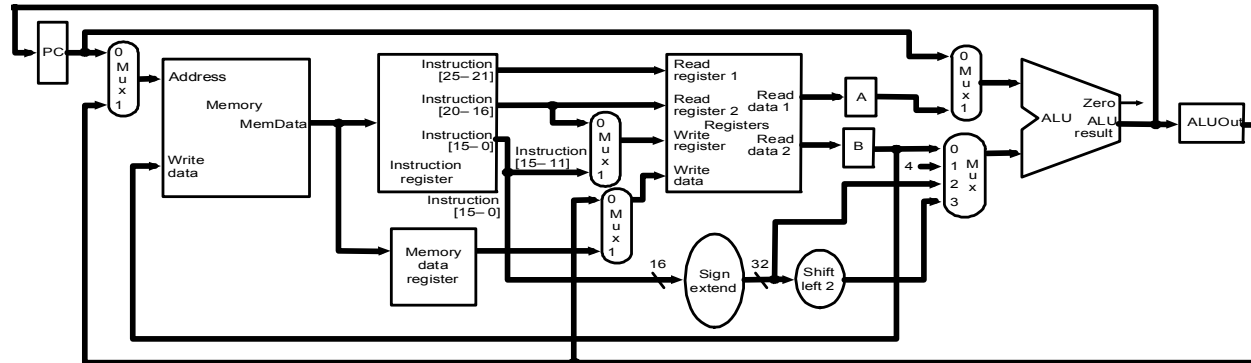
```
A = Reg[IR[25-21]] ;  
B = Reg[IR[20-16]] ;  
ALUOut = PC + (sign-extend(IR[15-0]) << 2) ;
```

- Note que a operação está sendo decodificada



Passo 3: Depende da Instrução

- ULA executa um dos três casos abaixo com base na instrução
- Referência a Memória
$$\text{ALUOut} = A + \text{sign-extend}(\text{IR}[15-0]);$$
- Tipo-R
$$\text{ALUOut} = A \text{ op } B;$$
- Desvio:
$$\text{if } (A == B) \text{ PC} = \text{ALUOut};$$



Passo 4: Depende da Instrução

- Referência a Memória

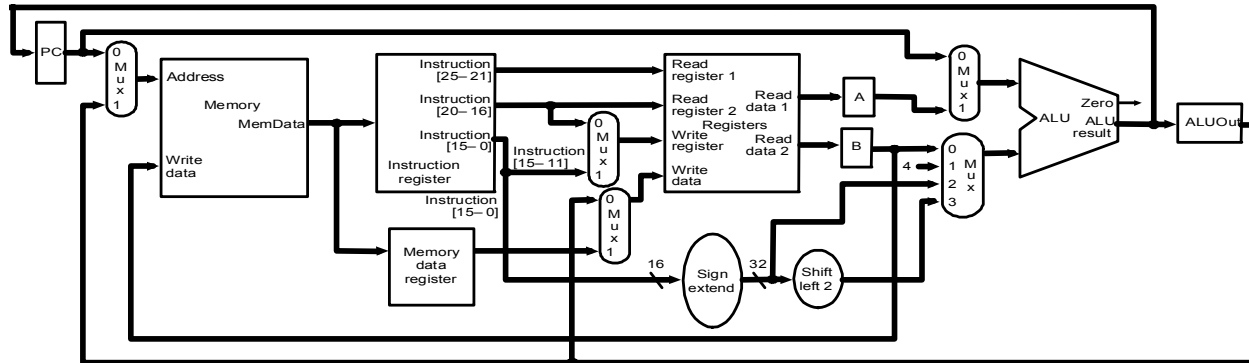
`MDR = Memory[ALUOut];`

or

`Memory[ALUOut] = B;`

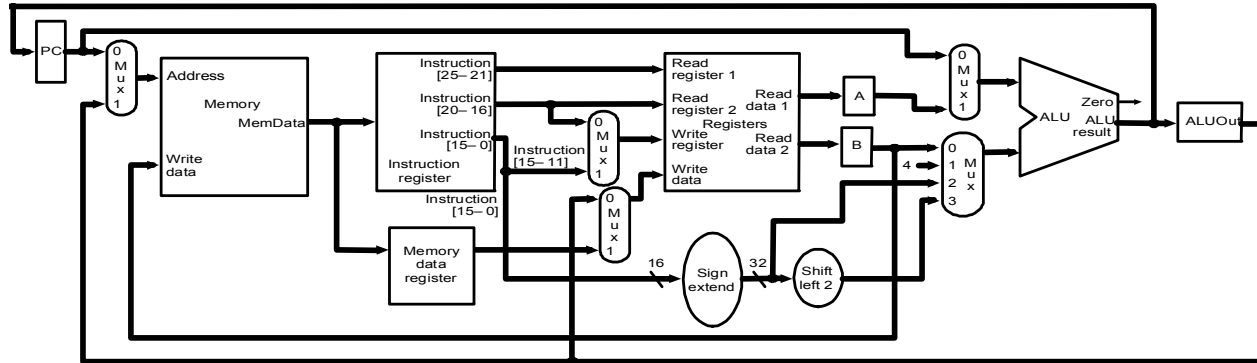
- Tipo-R

`Reg[IR[15-11]] = ALUOut;`



Passo 5: Escrita em Registrador

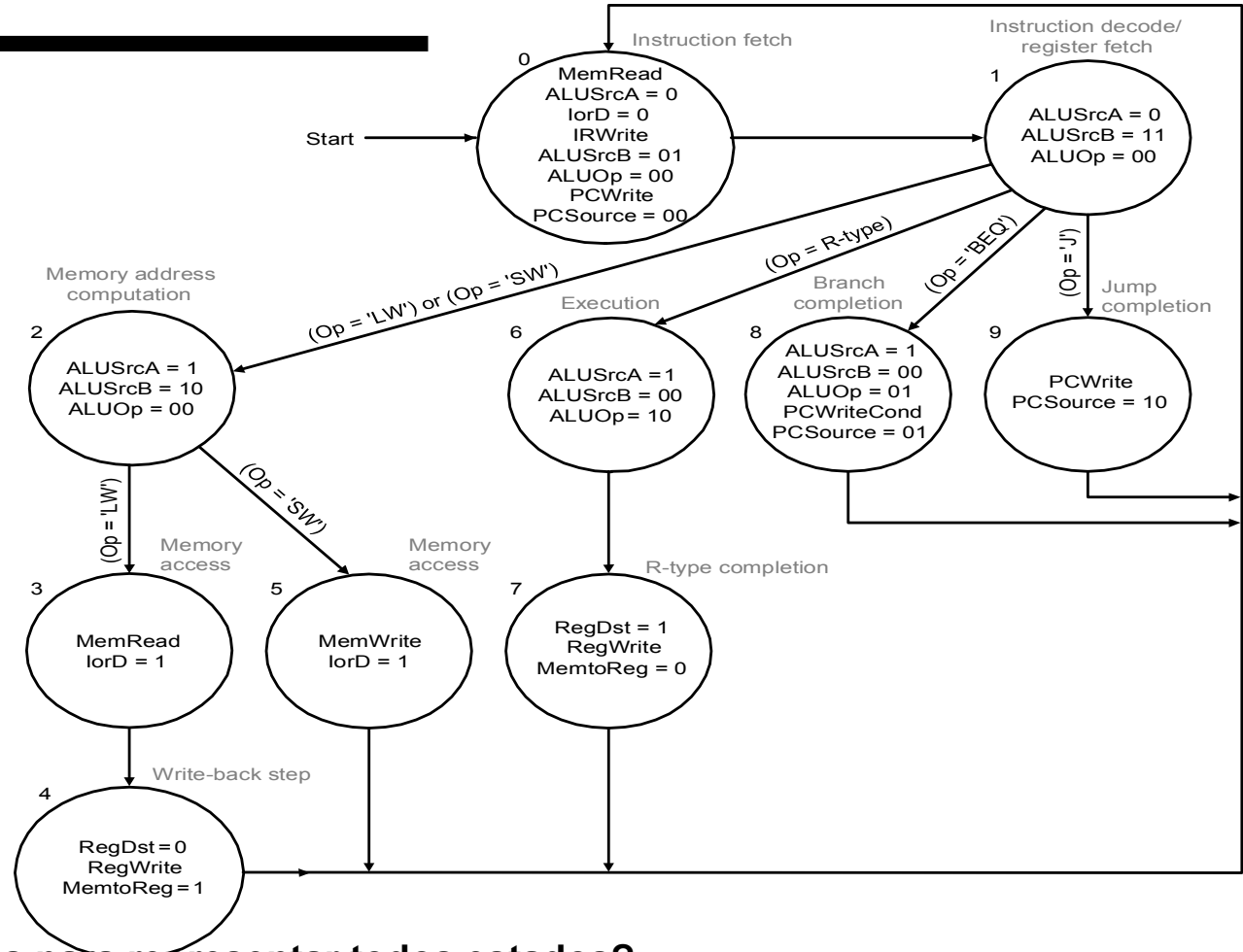
- $\text{Reg}[\text{IR}[20-16]] = \text{MDR};$



Resumo:

Step name	Action for R-type instructions	Action for memory-reference instructions	Action for branches	Action for jumps
Instruction fetch	IR = Memory[PC] PC = PC + 4			
Instruction decode/register fetch	A = Reg [IR[25-21]] B = Reg [IR[20-16]] ALUOut = PC + (sign-extend (IR[15-0]) << 2)			
Execution, address computation, branch/ jump completion	ALUOut = A op B	ALUOut = A + sign-extend (IR[15-0])	if (A ==B) then PC = ALUOut	PC = PC [31-28] (IR[25-0]<<2)
Memory access or R-type completion	Reg [IR[15-11]] = ALUOut	Load: MDR = Memory[ALUOut] or Store: Memory [ALUOut] = B		
Memory read completion		Load: Reg[IR[20-16]] = MDR		

Máquina de Estados



- Quantos bits são necessários para representar todos estados?

Exemplo:

- Quantos ciclos são necessários para executar o código abaixo?

```
lw $t2, 0($t3)
lw $t3, 4($t3)
beq $t2, $t3, Label
add $t5, $t2, $t3
sw $t5, 8($t3)
```

← #assume não

Label: ...

- O que está acontecendo durante o oitavo ciclo?
- Em qual ciclo ocorre de fato a adição entre \$t2 e \$t3?



Problema 3

Controle Multiclo: especifique uma implementação multiclo de uma máquina com as seguintes instruções:

- **“Add \$r1, 8(\$r2)”** que soma o conteúdo do registrador \$r1 com o conteúdo da memória na posição $8 + \$r2$ e salva na mesma posição, isto é, $\text{Mem}[\$r2+8] = \$r1 + \text{Mem}[\$r2+8]$
- **“Add 6(\$r1), 8(\$r2)”** que significa $\text{Mem}[\$r2+8] = \text{Mem}[\$r1+6] + \text{Mem}[\$r2+8]$.

Quanto ciclos são necessários para executar cada uma das instruções?